Laboratorio 2026

Carnevali, Alessandro 2061348

Fazzello, Simone 2071633

Sandri, Filippo 2108387

1 Obiettivo del progetto:

L'obiettivo del progetto è sviluppare una **pipeline** automatizzata per **raccogliere**, **ripulire** e **valutare** la qualità di testi estratti dal web, orientata all'uso dei dati raccolti da parte degli **LLM**. Il sistema, orchestrato in **multi-container Docker**, offre un'architettura scalabile accessibile sia tramite **Web UI** che **REST API**, facilitando così la gestione e il testing dell'intero flusso di elaborazione dati.

2 Organizzazione del codice:

Il codice è diviso in due microservizi principali: `Frontend` e `Backend`, e lo pseudo-database `Gs_data`, che viene montato come volume esterno nel microservizio di backend.

2.1 Backend

Backend è dove risiede la logica del parser. Le componenti più importanti del microservizio sono il **Crawler** e il **Server API**

2.1.1 Crawler

per effettuare il **parsing** e lo **scraping** delle fonti web abbiamo usato la libreria open-source **Crawl4ai**, ottimizzata per l'estrazione di dati per l'AI.
Tra le varie strategie di parsing che la libreria offre abbiamo scelto di usare
JsonCssExtractionStrategy, una strategia che usa i selettori CSS per trasformare il file html in un file json strutturato con solo gli elementi considerati importanti. Questo selettore è stata per noi la scelta più sensata perché è una strategia molto leggera e veloce e lo scraping ottenuto in questo modo è di ottima qualità (***F1 +-= 0.95***).
Oltre al parsing di Crawl4ai in alcuni casi è stato usato un **post-processing** per pulire ancora di più l'output, ad esempio per eliminare citazioni o caratteri superflui.

Dal punto dell'implementazione del crawler si è usata la logica della **programmazione orientata ad oggetti** (OOP): con una classe base `Crawler()` e le sottoclassi `WikipediaCrawler()`, `BorsaItalianaCrawler()`, `EuropeanCouncilCrawler()` e `TradingEconomicsCrawler()` per i diversi domini.

2.1.2 Server API

Il Server API rappresenta il punto di accesso principale al microservizio di backend ed è stato sviluppato tramite **FastAPI**, framework Python particolarmente adatto alla realizzazione di servizi **REST**.

La responsabilità del Server API è esporre verso l'esterno le funzionalità principali del parser, portando la logica del crawler e della valutazione all'interno di una serie di endpoint. In particolare, il server si occupa di ricevere le richieste provenienti dal frontend o da client esterni, verificare e validare i dati in ingresso e instradare correttamente le operazioni richieste.

Gli endpoint implementati permettono di:

- ottenere la lista dei domini supportati;
- richiedere il parsing di una pagina web;
- richiedere il gold standard associato a una pagina web;
- eseguire la valutazione dell'output estratto;
- restituire i risultati sotto forma di risposta JSON.

Dal punto di vista architetturale, il Server API svolge quindi il ruolo di strato intermedio tra il crawler e i client esterni.

2.2 Frontend

In Frontend sono presenti gli endpoint API del client e il codice della **web UI**.

2.2.1 Frontend API

Nel microservizio Frontend è presente un livello dedicato alla comunicazione con il backend, **Frontend API**. Tale componente ha la responsabilità di gestire tutte le richieste HTTP verso il Server API, fungendo da intermediario tra l'interfaccia utente e la logica applicativa.

In particolare, questo livello si occupa di:

- costruire le richieste verso gli endpoint del backend;
- inoltrare parametri come URL, dominio o testi da valutare;
- ricevere e interpretare le risposte JSON del server;
- gestire eventuali errori di comunicazione o input non validi;
- fornire alla Web UI dati per essere mostrati all'utente.

Separare la logica di chiamata alle API dalla parte puramente visuale dell'interfaccia consente alla Web UI di evitare di conoscere i dettagli di basso livello della comunicazione HTTP, ma può limitarsi a invocare le funzioni del client frontend e a mostrare i risultati ricevuti.

Questa scelta progettuale rende inoltre più semplice la manutenzione del sistema: eventuali modifiche agli endpoint del backend o agli indirizzi di rete possono essere gestite in un solo punto, senza dover intervenire direttamente sull'intera interfaccia utente.

2.2.2 Web UI

è stata realizzata una *ui* interattiva per interagire con l'applicazione. É possibile la visione dei risultati del parser, del gold standard di reiferimento e delle metriche di valutazione del parser oltre alla sottomissione di url qualsiasi per ottenere del testo parsato (purchè appartenenti ai domini assegnati). É anche possibile visionare i *file di log* per il backend e il frontend. è scritto in **HTML** e **Java script**, è stato usato **Jinja2** per rendere la pagina dinamica.

2.3 Gs_data

nella cartella `Gs_data` sono presenti i file .json del **gold standard**, usati per valutare il parser attraverso il confronto con dei testi corretti estratti in modo manuale. Per ogni dominio supportato è presente un file .json popolato con 5 url specifici del dominio. Nel nostro caso abbiamo quindi i seguenti file: `gs_borsaitaliana.json`, `gs_consiliumeuropa.json`, `gs_tradingeconomics.json` e `gs_wikipedia.json`

2.4 Containerizzazione

Per garantire l'isolamento dell'ambiente di esecuzione e la riproducibilità del software, il progetto utilizza **Docker** per la containerizzazione. Nello specifico, l'architettura è stata suddivisa in due container: *frontend* e *backend*, ognuno corredato dal proprio `Dockerfile`.

L'orchestrazione di questi container è gestita tramite **Docker Compose**, che permette di avviare l'intera applicazione attraverso un singolo file di configurazione in formato YAML (`docker-compose.yml`).

3 Valutazione parser:

I parser sono stati valutati tramite le seguenti metriche obbligatorie:

- **Precision** = $\frac{|tokens_estratti \cap tokens_gs|}{|tokens_estratti|}$. Controlla, tra i token trovati dal modello, quanti sono corretti.
- **Recall** = $\frac{|tokens_estratti \cap tokens_gs|}{|tokens_gs|}$. Controlla, tra tutti i token corretti possibili, quanti il modello è riuscito a trovare.
- **F1** = $2 \times \frac{precision \times recall}{precision + recall}$. Rappresenta la media armonica tra precision e recall.

Inoltre sono state ritenute utili per la valutazione dell'operato del parser altre due metriche:

- **Levenshtein distance**. Un algoritmo che indica il numero minimo di modifiche necessarie per rendere le due stringhe identiche.
- **Jaccard Similarity** = $\frac{|tokens_estratti \cap tokens_gs|}{|tokens_estratti \cup tokens_gs|}$. Mostra quanto sono simili i due insiemi

Tutte le metriche usate sono normalizzate per mostrare un valore da 0 a 1.00, dove 1.00 è il punteggio perfetto.